

# 결측치 제거와 대체

버릴 것과 채울 것을 가르는 기준을 배웁니다

dropna thresh / subset fillna ffill / bfill transform

2026년 8월 21일 (금) · 23일차 · 56차시

교안 15\_02 결측치 제거와 대체 — 실습 1~9

국립금오공과대학교 컴퓨터공학부 박민호

| 구분             | 내용                                | 상태     |
|----------------|-----------------------------------|--------|
| PART 01 결측치 제거 | dropna 네 옵션 · 비율 기준 컬럼 제거 · 손실 비교 | 완주     |
| PART 02 결측치 대체 | 평균 · 중앙값 · 최빈값 · 앞뒤 값 · 그룹별       | 완주     |
| 실습 1~9         | 제거·대체 비교부터 파일 저장까지                | 완주     |
| 다음 회차          | 16_01 이상치 개념과 사분위수                | 8월 24일 |

# 이 책을 읽는 법

뱃지 · 코드블록 · 이전 회차와의 연결

## 1. 분류 뱃지

모든 항목 이름 앞에는 그것이 **무엇인지**를 알려주는 색 뱃지가 붙습니다. 이름만 외우면 나중에 반드시 헛갈립니다.

| 뱃지  | 무엇                  | 판별 신호            | 오늘의 예                                                       |
|-----|---------------------|------------------|-------------------------------------------------------------|
| 메서드 | 객체에 붙는 함수           | 점 뒤에 있고 괄호가 붙습니다 | <code>.dropna()</code> <code>.fillna()</code>               |
| 속성  | 객체가 가진 값            | 점 뒤에 있고 괄호가 없습니다 | <code>.shape</code> <code>.index</code>                     |
| 키워드 | 함수에 넘기는 이름<br>붙은 인자 | 등호 왼쪽에 옵니다       | <code>thresh=</code> <code>subset=</code> <code>how=</code> |
| 개념  | 문법이 아닌 사고 틀         | 코드로 쓸 수 없습니다     | 결측률 · 손실률 · 대체 전략                                           |
| 패턴  | 자주 쓰는 코드 흐름         | 여러 요소의 조합입니다     | <code>isna().sum() / len(df)</code>                         |

### 개념

**3초 판별법.** ① 앞에 점(.)이 있습니까? 없으면 함수나 키워드입니다. ② 점이 있다면 뒤에 괄호가 붙습니까? 붙으면 메서드, 안 붙으면 속성입니다. 오늘 자주 쓸 `na_rate[na_rate > 0.4].index.tolist()`를 이 기준으로 읽으면 `.index`는 속성, `.tolist()`는 메서드입니다. 한 줄에 붙어 있어 헛갈리기 쉬운 자리입니다.

## 2. 코드블록 읽는 법

회색 배경에 왼쪽 파란 바가 붙은 상자는 **파이썬 코드**입니다. 상자 아래 가로줄 밑에 초록색 ▶로 시작하는 줄은 **그 코드를 실제로 실행했을 때 나온 출력**입니다. 이 책의 모든 출력은 실제 실행 결과를 옮긴 것입니다.

예시

```
import pandas as pd
```

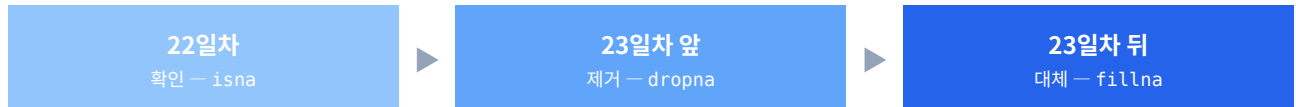
```
df = pd.read_csv("Data/15_02_사출성형_공정.csv", encoding="utf-8")
print(df.shape, "→", df.dropna().shape)
```

▶ (250, 22) → (76, 22)

본문에서 이렇게 회색으로 칠해진 글자는 코드 조각입니다. **굵은 글씨**는 꼭 붙잡아야 할 문장입니다.

### 3. 이전 회차와의 연결

어제(22일차)는 **결측이 어디에 얼마나 있는지 확인**했습니다. 오늘은 그 확인 결과를 가지고 **실제로 처리**합니다. 어제 배운 "확인 → 판단 → 처리 → 재확인" 고리에서 오늘은 3단계와 4단계에 해당합니다.



#### 연결

어제 계산한 **결측률 40% 기준선**이 오늘 실제 코드가 됩니다. 그리고 20일차·21일차에 배운 **평균과 중앙값의 차이**가 "무엇으로 채울까"라는 실전 판단으로 이어집니다. **이미 배운 것들이 오늘 전부 도구로 쓰입니다.** 새로 외울 것은 dropna와 fillna의 옵션 이름 정도입니다.

## CONTENTS

# 목차

오늘 배우는 것을 한눈에

| 장      | 제목                | 무엇을 배우나                          |
|--------|-------------------|----------------------------------|
| CH 0   | 오늘의 지도            | 제거와 대체 — 정반대의 두 접근               |
| STEP 1 | dropna 네 가지 옵션    | how · thresh · subset으로 삭제 기준 조절 |
| STEP 2 | 비율로 컬럼 자르고 손실 재기  | 실습 3·4 — 40% 기준과 손실을 비교          |
| STEP 3 | fillna — 무엇으로 채울까 | 평균 · 중앙값 · 최빈값 · 앞뒤 값 · 그룹별      |
| STEP 4 | 제거 대 대체, 그리고 저장   | 실습 8·9 — 보존율 비교와 파이프라인           |
| 부록 A   | 치트시트              | 오늘 나온 모든 도구를 한 표에                |
| 부록 B   | 오류·함정 사전          | 자주 걸리는 함정 · 교안 대조                |
| 부록 C   | 실습 하이라이트와 다음 시간   | 핵심 결론 정리 · 이상치 단위 예고             |

### 개념

오늘의 한 문장. **"삭제는 정직하지만 비싸고, 대체는 값싸지만 추측이다."** 결측이 있는 행을 다 지우면 남은 데이터는 전부 진짜 측정값입니다. 대신 이 데이터에서는 **70%가 사라집니다**. 채우면 250행을 다 지키지만, 그중 상당수는 **실제로 측정된 적 없는 값**입니다. 어느 쪽도 공짜가 아닙니다.

## 오늘의 지도

제거와 대체 — 정반대의 두 접근

## 0-1. 처리의 두 갈래

| 구분     |                          |                          |
|--------|--------------------------|--------------------------|
| 작동 방식  | 결측이 있는 행·열을 통째로 삭제       | 빈 칸만 콕 집어 그럴듯한 값으로 채움    |
| 데이터 크기 | <b>줄어들</b>               | <b>원래 크기 유지</b>          |
| 장점     | 추측이 없어 정직 · 왜곡 없음        | 데이터를 하나도 안 버림            |
| 단점     | 멀쩡한 값까지 함께 손실            | 채운 값은 결국 추측              |
| 판다스 도구 | <code>df.dropna()</code> | <code>df.fillna()</code> |

## 개념

둘 중 하나만 고르는 것이 아닙니다. **실전에서는 섞어 씁니다**. 결측이 절반 넘는 컬럼은 제거하고, 나머지는 대체로 채우는 식입니다. 오늘 실습 9가 정확히 그 흐름입니다. **어느 컬럼을 제거하고 어느 컬럼을 채울지 가르는 기준을** 만드는 것이 오늘 배울 전부입니다.

## 0-2. 오늘 다루는 데이터

| 파일                | 행 × 열    | 성격              | 결측             |
|-------------------|----------|-----------------|----------------|
| 15_02_사출성형_공정.csv | 250 × 22 | 사출성형 공정 250 사이클 | 12개 컬럼에 총 434개 |

어제 쓴 15\_01\_사출성형\_공정.csv와 **컬럼 순서만 다르고 구조는 같습니다**. 결측이 뒤쪽 컬럼일수록 많아지는 계단 모양도 그대로입니다. 다만 어느 컬럼이 어디에 놓였는지가 달라 **결측률 순위의 이름이 바뀝니다**.

오늘 데이터의 결측 지도

```
import pandas as pd
```

```
df = pd.read_csv("Data/15_02_사출성형_공정.csv", encoding="utf-8")  
na = df.isna().sum()  
print(na[na > 0].sort_values(ascending=False))
```

```
▶ 감압시간    109  
▶ 최대사출속도 109  
▶ 전환압력    68  
▶ 최대사출압   60  
▶ 계량시작점   34  
▶ 최소쿠션     34  
▶ 계량시작위치 34  
▶ 전환위치     9  
▶ 계량시간     9  
▶ 스크루위치   5  
▶ 사출압력     3  
▶ 스크루속도   1  
▶ dtype: int64
```

#### 설비 적용

**사출성형**은 플라스틱 원료를 녹여 금형에 밀어 넣고 굳혀 제품을 만드는 공정입니다. 한 사이클이 약 16.6초이고 250번의 사이클이 담겨 있습니다. 결측이 많은 **감압시간·최대사출속도**는 사이클의 **마지막 단계**에서 측정되는 항목이라, 사이클이 중간에 끊기면 통째로 기록되지 않습니다. 어제 확인한 대로 **109건이 정확히 같은 행에서 함께** 있습니다.

### 0-3. 오늘 등장하는 옵션과 용어

| 이름            | 한 줄 정의                             | 어디서    |
|---------------|------------------------------------|--------|
| how=          | any(하나라도) vs all(전부) — 삭제 기준       | STEP 1 |
| thresh=       | <b>살아 있는 값</b> 의 최소 개수 (결측 개수가 아님) | STEP 1 |
| subset=       | 지정한 컬럼만 보고 결측 판단                   | STEP 1 |
| numeric_only= | 글자 컬럼을 빼고 숫자 컬럼만 계산                | STEP 4 |
| .transform()  | 그룹별 계산 결과를 원래 행 위치로 되돌리기           | STEP 3 |
| 손실률           | 원본 대비 잃은 행의 비율 — <b>1 - 남은비율</b>   | STEP 2 |

| 이름  | 한 줄 정의         | 어디서    |
|-----|----------------|--------|
| 보존율 | 원본 대비 남은 행의 비율 | STEP 4 |

## 0-4. 처리 전에 반드시 던지는 두 질문

도구를 쓰기 전에 물어야 할 것이 둘 있습니다. 이 두 질문이 오늘 모든 판단의 출발점입니다.

| 질문       | 왜 묻나                               | 답이 바뀌면                |
|----------|------------------------------------|-----------------------|
| 왜 비었는가   | 센서 고장인지 통신 끊김인지 정답 미기입인지           | 정답 컬럼이면 채우지 말고 그 행을 뺌 |
| 얼마나 비었는가 | 같은 109개도 250행이면 43.6%, 4만 행이면 0.3% | 비율에 따라 제거·대체가 갈림      |

### 개념

첫 질문에 답하려면 **데이터 밖의 지식**이 필요합니다. 어제 배운 대로 결측의 원인은 센서 고장·점검 교체·환경 요인·통신 끊김·표 합치기로 갈립니다. 그런데 데이터만 봐서는 어느 쪽인지 알 수 없습니다. 두 번째 질문은 코드로 답할 수 있고, 그 답이 곧 **처리 방향**입니다.

그리고 어제 배운 한 가지를 다시 짚고 갑니다. **정답 컬럼이 빈 행은 채우면 안 됩니다.** 입력값이 비면 추측해서 채워도 되지만, 정답을 추측해 채우면 **모델이 틀린 것을 외웁니다.** 오늘 배울 `subset=` 옵션이 정확히 이 상황을 위한 것입니다.

### 주의

오늘 가장 헷갈리는 것이 `thresh`입니다. **결측이 몇 개 이하인가가 아니라 값이 몇 개 이상인가**를 정합니다. 22개 컬럼에서 `thresh=20`은 "값이 20개 이상 있는 행만 남겨라"이고, 뒤집으면 "결측이 2개 이하인 행만 남겨라"입니다. **방향을 반대로 이해하기 쉬우니 늘 한 번 더 확인하십시오.**

## STEP 1

# dropna 네 가지 옵션

how · thresh · subset으로 삭제 기준 조절

어제 개념만 짚고 넘어간 `dropna()`의 본론입니다. 옵션 없이 쓰면 너무 많이 지워지므로, **얼마나 지울지 조절하는 손잡이**가 네 개 붙어 있습니다.

## 1-1. 옵션 없는 `dropna` — 70%가 사라진다

**메서드** `.dropna()` 결측이 있는 행을 통째로 삭제

**정의** 결측이 **하나라도** 있는 행을 전부 지웁니다. 옵션을 주지 않으면 이것이 기본 동작입니다.

**형태** `df.dropna()` | `df.dropna(axis=1)`

```
import pandas as pd

df = pd.read_csv("Data/15_02_사출성형_공정.csv", encoding="utf-8")
print(df.shape)

# 결측 있는 행을 모두 삭제
print(df.dropna().shape)

# 방향을 열로 바꿔 결측 있는 열을 삭제
print(df.dropna(axis=1).shape)
```

---

- ▶ (250, 22)
- ▶ (76, 22) ← 174행이 사라짐
- ▶ (250, 10) ← 12개 컬럼이 사라짐

**왜 쓰나** 가장 단순하고 정직한 처리입니다. **추측이 없으니 왜곡도 없습니다.** 남은 76행은 전부 진짜 측정값입니다.

**응용** 문제는 대가입니다. **250행이 76행이 되면 70% 손실**입니다. 제출 코드의 주석이 심정을 잘 표현했습니다. **"이 정도면 3분의 2 이상이 날아가 버림."** 열 방향도 마찬가지로 22열이 10열이 됩니다.

```
# 얼마나 잃는지 숫자로
```

```
print("행 손실률:", round((1 - len(df.dropna()) / len(df)) * 100, 1), "%")  
print("열 손실률:", round((1 - df.dropna(axis=1).shape[1] / df.shape[1]) * 100, 1), "%")
```

```
▶ 행 손실률: 69.6 %
```

```
▶ 열 손실률: 54.5 %
```

## 개념

axis=1은 어제 배운 그대로 **가로 방향**입니다. sum(axis=1)이 행마다 계산했듯, dropna(axis=1)은 열을 지웁니다. 헛갈리면 **"axis는 없애는 축"**을 떠올리십시오.

## 자주 하는 실수

df.dropna()는 **원본을 바꾸지 않습니다**. 새 데이터프레임을 만들어 돌려줄 뿐입니다. 반드시 clean = df.dropna()처럼 변수에 담으십시오. 담지 않으면 화면에는 처리된 듯 보이지만 원본은 그대로입니다.

## 1-2. how — 어느 하나라도 vs 전부

**키워드** how= 삭제 기준을 any와 all 중에서 고르기

**정의** "any"(기본값)는 결측이 **하나라도** 있으면 삭제, "all"은 **모든 값이** 결측일 때만 삭제합니다.

**형태** df.dropna(how="all")

```
# 완전히 빈 행만 삭제  
print(df.dropna(how="all").shape)
```

```
▶ (250, 22) ← 하나도 안 지워짐
```

**왜 쓰나** how="all"은 **완전히 빈 행만 걷어내는 안전한 첫 청소**입니다. 엑셀에서 옮겨 온 데이터에는 아무것도 없는 빈 줄이 섞이기 쉬운데, 그것만 골라 지웁니다.

**응용** 이 데이터에서는 250행이 그대로 남았습니다. 제출 코드의 해석이 정확합니다. **"NaN으로 모든 컬럼 내용이 다 채워진 row가 없다는 뜻."** 즉 **완전히 빈 행은 하나도 없습니다**.

```
# 정말 없는지 직접 확인 — 행별 결측 수의 최댓값
```

```
print("한 행이 가진 최대 결측 수:", df.isna().sum(axis=1).max(), "/ 전체 열:", df.shape[1])
```

```
▶ 한 행이 가진 최대 결측 수: 8 / 전체 열: 22
```

## 개념

가장 부실한 행도 22개 중 8개만 비었습니다. 그러니 `how="all"`로는 아무것도 안 지워지는 것이 당연합니다. 결과가 0건이라고 코드가 틀린 것이 아닙니다. 어제 배운 행별 결측 세기가 이 확인에 쓰입니다.

### 1-3. thresh — 살아 있는 값의 최소 개수

**키워드** `thresh=` 값이 N개 이상 있는 행만 남기기

**정의** 결측 개수가 아니라 살아 있는 값의 최소 개수를 정합니다. 방향을 반대로 이해하기 쉬운 옵션입니다.

**형태** `df.dropna(thresh=20)`

```
# 값이 20개 이상인 행만 남기기
print(df.dropna(thresh=20).shape)

▶ (162, 22)
```

**왜 쓰나** `dropna()`의 70% 손실은 너무 큼니다. `thresh`를 쓰면 아주 부실한 행만 골라 지우고 나머지는 살릴 수 있습니다. 손실이 69.6%에서 35.2%로 절반이 됩니다.

**응용** 22개 컬럼에서 `thresh=20`은 값이 20개 이상, 즉 결측이 2개 이하인 행만 남긴다는 뜻입니다. 제출 코드의 해석이 정확합니다. "**250 - 162 = 88개 row는 NaN이 3개 이상이라는 뜻.**"

```
# thresh를 바꿔 가며 몇 행이 남는지
for t in [15, 18, 20, 21, 22]:
    n = len(df.dropna(thresh=t))
    print(f"thresh={t}: {n}행 ({round((1-n/250)*100, 1)}% 손실)")

▶ thresh=15: 249행 (0.4% 손실)
▶ thresh=18: 223행 (10.8% 손실)
▶ thresh=20: 162행 (35.2% 손실)
▶ thresh=21: 121행 (51.6% 손실)
▶ thresh=22: 76행 (69.6% 손실)
```

## 개념

`thresh=22`가 곧 `dropna()`와 같습니다. 컬럼이 22개니 "값이 22개 다 있어야 산다"는 말은 "결측이 하나도 없어야 산다"와 같은 말입니다. 이렇게 양 끝을 확인해 보면 옵션의 의미가 확실해집니다.

## 주의

thresh는 **비율이 아니라 개수**입니다. thresh=0.9처럼 비율을 넣으면 원하는 대로 동작하지 않습니다. 컬럼 수가 바뀌면 넣을 숫자도 바뀌므로, 실무에서는 `thresh=int(df.shape[1] * 0.9)`처럼 계산해서 넣는 편이 안전합니다.

## 1-4. subset — 특정 컬럼만 기준으로

**키워드** subset= 지정한 컬럼만 보고 결측 판단

**정의** 목록에 적은 **그 컬럼들만** 확인해서 삭제를 결정합니다. 다른 컬럼이 비어 있어도 무시합니다.

**형태** `df.dropna(subset=["불량여부"])`

```
# 불량여부 컬럼이 빈 행만 삭제
print(df.dropna(subset=["불량여부"]).shape)

▶ (250, 22)   ← 하나도 안 지워짐
```

**왜 쓰나** 정답 컬럼을 지키는 용도가 대표적입니다. 지난 시간에 배운 대로 입력값 결측은 채워서 쓸 수 있지만 **정답이 빈 행은 채우면 안 됩니다**. 가짜 정답을 가르치면 모델이 틀린 걸 외우기 때문입니다. 그래서 정답 컬럼만 기준으로 삼아 그 행만 걷어냅니다.

**응용** 이 데이터에서는 결과가 250행 그대로입니다. 제출 코드의 해석대로 **"불량여부 컬럼에는 NaN이 하나도 없다"**는 뜻입니다. 여러 컬럼을 함께 넣을 수도 있고, 그러면 **그중 하나라도 비면** 삭제됩니다.

```
# 결측이 많은 컬럼을 기준으로 넣어 보면 차이가 보인다
print("불량여부 기준   :", df.dropna(subset=["불량여부"]).shape)
print("감압시간 기준   :", df.dropna(subset=["감압시간"]).shape)
print("감압+최대사출속도 :", df.dropna(subset=["감압시간", "최대사출속도"]).shape)

▶ 불량여부 기준   : (250, 22)
▶ 감압시간 기준   : (141, 22)   ← 109건 삭제
▶ 감압+최대사출속도 : (141, 22)   ← 둘이 같은 행에서 빈
```

## 개념

세 번째 결과가 두 번째와 **똑같이 141행**입니다. 어제 발견한 **동시 결측**이 여기서 다시 확인됩니다. 감압시간과 최대사출속도는 109건이 전부 같은 행에서 비기 때문에, 둘을 함께 넣어도 지워지는 행이 늘지 않습니다.

## 1-5. 네 옵션 비교표

| 옵션           | 기준             | 이 데이터 결과   | 언제 쓰나            |
|--------------|----------------|------------|------------------|
| 없음           | 결측 하나라도 있으면 삭제 | 250 → 76행  | 손실 너무 큼          |
| axis=1       | 결측 있는 열 삭제     | 22 → 10열   | 컬럼이 많고 쓸 것만 남길 때 |
| how="all"    | 모든 값이 빈 행만 삭제  | 250 → 250행 | 안전한 첫 청소         |
| thresh=20    | 값이 20개 이상인 행만  | 250 → 162행 | 실전에서 가장 유용       |
| subset=[...] | 지정 컬럼만 보고 판단   | 250 → 250행 | 정답 컬럼 보호         |

### 개념

네 옵션은 **함께 쓸 수 있습니다**. `df.dropna(subset=["불량여부"]).dropna(thresh=20)`처럼 이어 붙이면 **정답이 빈 행을 먼저 걷어낸 뒤 부실한 행을 정리하는 두 단계**가 됩니다. 옵션 하나로 한 번에 끝내려 하지 말고, **단계를 나누면 각 단계에서 몇 행이 빠졌는지 확인할 수 있어 훨씬 안전합니다**.

그리고 네 옵션 모두 **원본을 바꾸지 않습니다**. 결과를 변수에 담지 않으면 아무 일도 일어나지 않습니다. 이 점이 결측 처리에서 가장 흔한 실수라 다시 강조합니다.

### 가장 흔한 실수

```
# x 화면에는 처리된 듯 보이지만 원본은 그대로
df.dropna(thresh=20)
print(df.shape)
```

```
# ✓ 반드시 변수에 담는다
clean = df.dropna(thresh=20)
print(clean.shape)
```

- ▶ (250, 22) ← 안 바뀜
- ▶ (162, 22) ← 제대로 반영

## STEP 2

# 비율로 컬럼 자르고 손실 재기

실습 3 · 4

어제 계산한 **결측률**을 실제 처리로 옮기는 단계입니다. 40%라는 기준선을 코드로 표현하고, 그 결과 얼마나 잃는지 표로 만듭니다.

## 2-1. 실습 3 — 40% 넘는 컬럼만 골라 제거

Practice/15\_02\_example.py — 실습 3

```
df_3 = pd.read_csv("Data/15_02_사출성형_공정.csv", encoding="utf-8")

# 컬럼별 결측 비율을 계산
na_rate_3 = df_3.isna().sum() / len(df_3)
print(na_rate_3[na_rate_3 > 0].round(3).sort_values(ascending=False))
```

```
▶ 감압시간    0.436
▶ 최대사출속도 0.436
▶ 전환압력    0.272
▶ 최대사출압   0.240
▶ 계량시작점   0.136
▶ 최소쿠션    0.136
▶ 계량시작위치 0.136
▶ 전환위치     0.036
▶ 계량시간     0.036
▶ 스크루위치   0.020
▶ 사출압력     0.012
▶ 스크루속도   0.004
```

### 개념

어제는 `isna().mean()`을 썼고 오늘은 `isna().sum() / len(df)`를 썼습니다. 두 방법은 완전히 같은 값을 줍니다. 참을 1로 세어 개수를 구한 뒤 전체로 나누는 것이 곧 평균이기 때문입니다. 강사님이 수업 중 "비율이란 말이 몇 %라고 없었으니까"라며 0.4로 표현하신 것도 같은 맥락입니다. 이 코드가 돌려주는 값은 0에서 1 사이의 소수이지 퍼센트가 아닙니다.

**패턴** 조건 필터 → index → tolist() 조건에 맞는 컬럼 이름만 목록으로

**정의** Series를 조건으로 거르면 그 index가 곧 컬럼 이름입니다. .tolist()로 파이썬 리스트로 꺼내면 그대로 drop()에 넘길 수 있습니다.

**형태** na\_rate[na\_rate > 0.4].index.tolist()

```
# 결측 비율 40%를 넘는 컬럼만 목록으로 뽑기
over_3 = na_rate_3[na_rate_3 > 0.4]
print(over_3.round(3))

drop_cols_3 = over_3.index.tolist()
print("제거 대상 컬럼:", drop_cols_3)

# 그 컬럼들을 drop으로 제거
df_dropped_3 = df_3.drop(columns=drop_cols_3)
print("제거 전:", df_3.shape, "-> 제거 후:", df_dropped_3.shape)
```

```
▶ 최대사출속도 0.436
▶ 감압시간 0.436
▶ dtype: float64
▶
▶ 제거 대상 컬럼: ['최대사출속도', '감압시간']
▶ 제거 전: (250, 22) -> 제거 후: (250, 20)
```

**왜 쓰나** 컬럼 이름을 손으로 타이핑하지 않아도 됩니다. 기준만 바꾸면 목록이 자동으로 다시 만들어집니다. 컬럼이 수백 개인 데이터에서는 이 방식이 유일한 선택입니다.

**응용** 기준을 바꿔 가며 몇 개가 걸리는지 보면 기준선의 의미가 눈에 들어옵니다.

```
for t in [0.05, 0.2, 0.3, 0.4, 0.5]:
    cols = na_rate_3[na_rate_3 > t].index.tolist()
    print(f"기준 {t}: {len(cols)}개 → {cols}")
```

```
▶ 기준 0.05: 7개 → 계량시작위치·계량시작점·최소쿠션·최대사출압·전환압력·최대사출속도·감압시간
▶ 기준 0.2 : 4개 → 최대사출압·전환압력·최대사출속도·감압시간
▶ 기준 0.3 : 2개 → 최대사출속도·감압시간
▶ 기준 0.4 : 2개 → 최대사출속도·감압시간
▶ 기준 0.5 : 0개 → 없음
```

## 개념

drop()에 columns=를 넘기면 열 방향 삭제입니다. axis=1을 쓰지 않아도 되니 훨씬 읽기 쉽습니다. 행을 지울 때는 df.drop(index=[...])입니다.

## 주의

`over_3.index.tolist()`에서 `.index`는 속성(괄호 없음), `.tolist()`는 메서드(괄호 있음)입니다. 한 줄에 둘이 붙어 있어 헷갈리기 쉬운데, 읽는 법에서 배운 3초 판별법이 여기서 쓰입니다.

## 2-2. 실습 4 — 손실률을 표로 만들기

강사님이 이 문제를 두고 "삭제 방식별로 남는 행의 수하고 손실된 비율을 표로 비교할 수 있게 해 달라"고 문제를 다시 읽어 주셨습니다. 핵심은 **표로 만드는 것**입니다.

Practice/15\_02\_example.py — 실습 4

```
rows_origin = len(df_4)
rows_dropna = len(df_4.dropna())
rows_thresh = len(df_4.dropna(thresh=20))

# 딕셔너리로 데이터프레임을 직접 만든다
compare_4 = pd.DataFrame({
    "방식": ["원본", "행삭제", "thresh20"],
    "행수": [rows_origin, rows_dropna, rows_thresh],
})
compare_4["손실률(%)"] = ((1 - compare_4["행수"] / rows_origin) * 100).round(2)
print(compare_4)
```

```
▶   방식  행수  손실률(%)
▶  0      원본   250      0.0
▶  1   행삭제    76   69.6
▶  2 thresh20   162   35.2
```

## 개념

강사님이 이 대목에서 **데이터프레임을 직접 만드는 법**을 따로 설명하셨습니다. "**데이터프레임 만들 때는 딕셔너리 방식을 사용합니다.**" `pd.DataFrame({"열이름": [값, 값, 값], ...})` 형태로, **키가 열 이름이 되고 리스트가 그 열의 값이 됩니다.** 어제 결측 비교표를 만들 때 쓴 것과 같은 문법입니다. **계산 결과를 표로 정리해 보고서에 붙이는 것이** 이 문법의 주 용도입니다.

그리고 손실률 계산식을 보십시오.  $(1 - \text{행수} / \text{원본행수}) * 100$ 입니다. **남은 비율을 1에서 빼서 잃은 비율로 뒤집는 것**입니다. 이런 작은 변환을 코드로 바로 쓸 수 있어야 표가 읽히는 물건이 됩니다.

### 주의

제출 코드의 결론이 정확합니다. "행삭제는 손실률 69.6%로 데이터의 3분의 2 이상이 날아감. thresh는 35.2%로 손실이 절반 수준이지만 여전히 큼. → 삭제만으로는 데이터를 지키기 어려우니 대체(fillna)가 필요." 이 한 문단이 다음 장으로 넘어가는 이유를 정확히 설명합니다. 표를 만든 목적은 표 자체가 아니라 이 결론입니다.

## 2-3. 표를 만들 때 자주 쓰는 두 문법

| 문법                               | 쓰임                 | 예                                        |
|----------------------------------|--------------------|------------------------------------------|
| <code>pd.DataFrame({...})</code> | 딕셔너리로 표를 직접 만들기    | <code>{"방식": [...], "행수": [...]}</code>  |
| <code>df["새열"] = 계산식</code>      | 없던 열 이름을 왼쪽에 쓰면 추가 | <code>df["손실률"] = (1 - a/b) * 100</code> |

두 문법 모두 오늘 여러 번 다시 나옵니다. **계산 결과를 표로 정리하는 일**이 전처리의 절반이기 때문입니다. 숫자를 화면에 흘뿌리는 것과 표 하나로 정리하는 것은 **보고서로서의 값어치**가 다릅니다.

## STEP 3

# fillna — 무엇으로 채울까

평균 · 중앙값 · 최빈값 · 앞뒤 값 · 그룹별

삭제로는 답이 안 나온다는 결론에 도달했으니 이제 **채웁니다**. 문제는 무엇으로 채우느냐입니다. 여기서 지난 시간에 배운 평균과 중앙값의 차이가 실전 판단이 됩니다.

## 3-1. 실습 5 — 평균과 중앙값 중 무엇으로

**메서드** `.fillna()` 결측을 지정한 값으로 채우기

**정의** 빈 칸만 꼭 집어 **괄호 안의 값으로 채웁니다**. 데이터 크기는 그대로 유지됩니다.

**형태** `df["열"].fillna(값)`

```
df_5 = pd.read_csv("Data/15_02_사출성형_공정.csv", encoding="utf-8")
print("NaN 개수:", df_5["최대사출압"].isna().sum())

# 두 값 모두 반드시 원본에서 구해야 함 (채운 뒤에 구하면 값이 오염됨)
mean_5 = df_5["최대사출압"].mean()
median_5 = df_5["최대사출압"].median()
print("mean():", round(mean_5, 3), "|median():", round(median_5, 3),
      "|차이:", round(mean_5 - median_5, 3))
```

```
▶ NaN 개수: 60
▶ mean(): 1241.672 |median(): 1238.405 |차이: 3.267
```

**왜 쓰나** 지난 시간에 배운 **"평균과 중앙값이 비슷하면 평균, 많이 다르면 중앙값"** 기준을 여기서 적용합니다. 차이가 3.267이고 값 자체가 1240 근처이니 **차이는 0.26% 수준**입니다. 사실상 같으므로 어느 쪽을 써도 무방한 컬럼입니다.

**응용** 제출 코드가 두 방식을 모두 만들어 `describe()`로 나란히 비교했습니다. 이 표에 **오늘 가장 중요한 관찰**이 숨어 있습니다.

```
s_fill_mean = df_5["최대사출압"].fillna(mean_5)
s_fill_median = df_5["최대사출압"].fillna(median_5)

compare_5 = pd.DataFrame({
    "원본": df_5["최대사출압"].describe(),
    "평균대체": s_fill_mean.describe(),
    "중앙값대체": s_fill_median.describe(),
}).round(3)
print(compare_5)
```

|       | 원본       | 평균대체     | 중앙값대체    |             |
|-------|----------|----------|----------|-------------|
| count | 190.000  | 250.000  | 250.000  |             |
| mean  | 1241.672 | 1241.672 | 1240.888 |             |
| std   | 8.512    | 7.415    | 7.546    | ← 편차가 줄었다   |
| min   | 1231.480 | 1231.480 | 1231.480 |             |
| 25%   | 1236.100 | 1236.630 | 1236.630 |             |
| 50%   | 1238.405 | 1240.840 | 1238.405 |             |
| 75%   | 1243.920 | 1241.672 | 1240.960 | ← 75%가 내려갔다 |
| max   | 1268.650 | 1268.650 | 1268.650 |             |

#### 개념

count가 190에서 250으로 늘었습니다. **결측 60건이 채워졌다는 증거**입니다. mean은 평균대체 쪽에서 원본과 정확히 같습니다. **평균으로 채우면 평균은 안 변합니다**. 당연한 이야기지만 코드로 확인해 두면 계산 도구가 됩니다.

#### 주의

주목할 것은 **표준편차가 8.512에서 7.415로 줄었다**는 점입니다. 그리고 **75% 분위수가 1243.92에서 1241.672로 내려갔습니다**. 60건이 한 값에 몰리면서 **분포가 인위적으로 뾰족해진 것**입니다. 어제 말한 **"채우면 분포가 뾰족해진다"**가 숫자로 나타난 자리입니다. 채운다고 공짜가 아닙니다.

## 3-2. 실습 6 — 범주형과 시계열

숫자가 아닌 컬럼에는 평균을 낼 수 없습니다. 그리고 시간 순서가 있는 데이터에는 더 자연스러운 방법이 있습니다.

**메서드** `.mode()` 가장 자주 나오는 값 (최빈값)

**정의** 가장 많이 등장한 값을 돌려줍니다. **결과가 Series라서 [0]으로 첫 값을 꺼내야 합니다**.

**형태** `df["열"].mode()[0]`

```
# 범주형 열의 최빈값을 구해 채우기
print("값 분포:")
print(df_6["사출기"].value_counts())
print("최빈값 mode()[0]:", df_6["사출기"].mode()[0])

df_6["사출기"] = df_6["사출기"].fillna(df_6["사출기"].mode()[0])
```

---

```
▶ 값 분포:
▶ 사출기
▶ 1호기 110
▶ 2호기 85
▶ 3호기 55
▶ Name: count, dtype: int64
▶
▶ 최빈값 mode()[0]: 1호기
```

**왜 쓰나** 글자·분류 데이터는 **평균을 낼 수 없습니다**. "1호기와 2호기의 평균"이라는 것은 존재하지 않습니다. 그래서 **가장 자주 나타난 값**으로 채웁니다.

**응용** mode()가 왜 Series를 돌려줄까요. **최빈값이 여러 개일 수 있기 때문**입니다. 두 값이 똑같이 100번씩 나왔다면 둘 다 최빈값이고, mode()는 둘 다 돌려줍니다. [0]은 그중 첫 번째를 고르는 것입니다.

```
# 최빈값이 둘인 경우
s = pd.Series(["A", "A", "B", "B", "C"])
print(s.mode())
print("첫 번째만:", s.mode()[0])
```

---

```
▶ 0    A
▶ 1    B
▶ dtype: object
▶
▶ 첫 번째만: A
```

### 자주 하는 실수

이 데이터의 사출기 컬럼은 **결측이 0건**입니다. 즉 채울 것이 없었습니다. 코드 자체는 맞지만 **동작을 확인하는 실습으로서는 아무 일도 일어나지 않은** 셈입니다. 실습을 돌린 뒤 "채우기 전 NaN 개수: 0"을 봤다면 그 사실을 짚고 넘어가야 합니다.

**메서드** .ffill() · .bfill() 앞 값 또는 뒤 값으로 채우기

**정의** ffill은 **바로 앞의 멀쩡한 값**으로, bfill은 **바로 뒤의 값**으로 빈 칸을 메웁니다. forward fill · backward fill의 줄임말입니다.

**형태** `df["열"].ffill().bfill()`

```
# 앞뒤 값으로 채우려면 시간 순서가 먼저 맞아야 한다
df_6 = df_6.sort_values("측정시각")

print("채우기 전 전환압력 NaN:", df_6["전환압력"].isna().sum())
df_6["전환압력"] = df_6["전환압력"].ffill().bfill()
print("채운 뒤 전환압력 NaN:", df_6["전환압력"].isna().sum())
```

- ▶ 채우기 전 전환압력 NaN: 68
- ▶ 채운 뒤 전환압력 NaN: 0

**왜 쓰나** 설비 상태는 짧은 시간에 급변하지 않습니다. 17초 전 압력이 1240이었다면 지금도 1240 근처일 가능성이 높습니다. 평균 하나로 다 채우는 것보다 **그 시점의 맥락**을 살립니다.

**응용** `ffill()`만 쓰면 **맨 앞 행이 결측일 때 채울 앞 값이 없어** 그대로 남습니다. 그래서 `bfill()`을 이어 붙여 마무리합니다. 순서를 바꿔 `bfill().ffill()`로 써도 되지만, 보통은 **과거 값으로 현재를 추측하는 편이 자연스러워** `ffill`을 먼저 씁니다.

```
s = pd.Series([None, None, 73.0, None, None, 75.0])
print("원본 :", s.tolist())
print("ffill :", s.ffill().tolist())
print("+bfill :", s.ffill().bfill().tolist())
```

- ▶ 원본 : [nan, nan, 73.0, nan, nan, 75.0]
- ▶ ffill : [nan, nan, 73.0, 73.0, 73.0, 75.0] ← 앞 두 칸이 안 채워짐
- ▶ +bfill : [73.0, 73.0, 73.0, 73.0, 73.0, 75.0]

## 개념

제출 코드가 `sort_values("측정시각")`을 **먼저** 한 것이 핵심입니다. **정렬이 안 된 상태에서 ffill을 쓰면 아무 관계 없는 행의 값을 끌어옵니다.** 시간 순서가 의미를 갖는 방법이므로 순서를 맞추는 것이 전제 조건입니다.

## 주의

결측이 **연달아 길게 이어질 때**도 주의해야 합니다. 68건 중 여러 개가 붙어 있으면 같은 값이 계속 반복됩니다. 위 예제에서도 73.0이 네 번 연속 들어갔습니다. **긴 결측 구간에는 ffill이 부적절합니다.**

### 3-3. 실습 7 — 그룹별로 채우기

전체 평균 하나로 다 채우면 **집단의 특성이 뭉개집니다**. 1호기와 3호기의 평소 값이 다르다면 각자의 평균으로 채우는 편이 정확합니다.

Practice/15\_02\_example.py — 실습 7

```
# 사출기로 그룹을 나눠 감압시간의 그룹별 평균 확인
print(df_7.groupby("사출기")["감압시간"].mean().round(6))
print("전체 평균:", round(df_7["감압시간"].mean(), 6))

# transform은 그룹별 계산 결과를 원래 행 위치에 그대로 돌려준다
df_7["감압시간"] = df_7.groupby("사출기")["감압시간"].transform(
    lambda s: s.fillna(s.mean())
)
print("채운 뒤 NaN:", df_7["감압시간"].isna().sum())
```

```
▶ 사출기
▶ 1호기  0.322179
▶ 2호기  0.322368
▶ 3호기  0.322400
▶ Name: 감압시간, dtype: float64
▶ 전체 평균: 0.32227
▶
▶ 채운 뒤 NaN: 0
```

#### 주의

그런데 이 데이터에서는 **그룹별 평균이 0.322179 · 0.322368 · 0.322400으로 사실상 같습니다**. 소수 넷째 자리까지 같습니다. 그러니 **그룹별로 채우나 전체 평균으로 채우나 결과가 거의 같습니다**. 기법은 배웠지만 이 데이터에서는 효과가 없는 셈입니다. **그룹별 대체는 그룹마다 평균이 실제로 다를 때만 의미가 있습니다**. 쓰기 전에 위 출력처럼 그룹별 평균을 먼저 찍어 보고 판단해야 합니다.

**메서드** `.transform()` 그룹별 계산 결과를 원래 행 위치로 되돌리기

**정의** `groupby` 뒤에 붙여 **그룹마다 계산한 결과를 원래 행 개수 그대로 돌려줍니다**. `agg()`가 그룹당 한 줄로 줄이는 것과 정반대입니다.

**형태** `df.groupby("기준")["열"].transform(함수)`

```
# agg는 줄이고, transform은 원래 길이를 유지한다
print("agg 결과 길이 :", len(df_7.groupby("사출기")["감압시간"].mean()))
print("transform 결과 길이:", len(df_7.groupby("사출기")
["감압시간"].transform("mean")))
print("원본 길이 :", len(df_7))
```

```
▶ agg 결과 길이 : 3
▶ transform 결과 길이: 250
▶ 원본 길이 : 250
```

**왜 쓰나** 결측을 채우려면 **원래 행 위치에 값이 놓여야** 합니다. `agg()`로 3줄짜리 결과를 받으면 250행에 다시 붙일 수가 없습니다. `transform()`이 그 붙이는 일을 대신 해 줍니다.

**응용** 괄호 안의 `lambda s: s.fillna(s.mean())`는 **이름 없는 함수**입니다. 여기서 `s`는 그룹 하나에 해당하는 Series이고, `s.mean()`은 **그 그룹의 평균**입니다. 그룹마다 이 함수가 한 번씩 실행됩니다.

```
# lambda 없이 풀어 쓰면 이런 뜻
def fill_with_group_mean(s):
    return s.fillna(s.mean())

df_7["감압시간"] = df_7.groupby("사출기")["감압시간"].transform(fill_with_group_mean)
```

## 개념

`transform("mean")`처럼 **문자열만 넘길 수도** 있습니다. 그러면 그냥 그룹 평균이 250행에 펼쳐집니다. 하지만 그것은 원본 값까지 평균으로 덮어써 버립니다. **결측만 채우려면 반드시 `fillna`를 포함한 함수를 넘겨야 합니다.**

## STEP 4

# 제거 대 대체, 그리고 저장

실습 8 · 9

결측치 단원의 마지막입니다. 지금까지 배운 두 갈래를 **같은 데이터에 나란히 적용해 비교**하고, 마지막으로 파일로 저장합니다.

## 4-1. 실습 8 — 같은 데이터, 두 가지 처리

Practice/15\_02\_example.py — 실습 8

```
df_8 = pd.read_csv("Data/15_02_사출성형_공정.csv", encoding="utf-8")

# 결측치 심한 컬럼을 먼저 뺀 기준 데이터
base_8 = df_8.drop(columns=["최대사출속도", "감압시간"])

# 제거 버전과 대체 버전을 각각 만든다
drop_8 = base_8.dropna()
fill_8 = base_8.fillna(base_8.median(numeric_only=True))

compare_8 = pd.DataFrame({
    "방식": ["기준", "제거판", "대체판"],
    "행수": [len(base_8), len(drop_8), len(fill_8)],
    "남은결측": [base_8.isna().sum().sum(), drop_8.isna().sum().sum(),
fill_8.isna().sum().sum()],
})
compare_8["보존율(%)"] = (compare_8["행수"] / len(base_8) * 100).round(1)
print(compare_8)
```

```
▶ 방식 행수 남은결측 보존율(%)
▶ 0 기준 250 434 100.0
▶ 1 제거판 110 0 44.0
▶ 2 대체판 250 0 100.0
```

숫자가 명확합니다. **제거판은 110행으로 44%만 남고, 대체판은 250행을 전부 지킵니다.** 부실 컬럼 두 개를 먼저 뺐는데도 제거판은 절반 이하로 줄었습니다.

## 주의

그렇다고 대체판이 무조건 낫다는 뜻은 아닙니다. 제출 코드의 마지막 주석이 균형을 잘 잡았습니다. **"대신 대체판은 채운 값이 실제 측정값이 아니라는 점을 감안해야 한다."** 250행이라고 해서 250개의 진짜 측정이 있는 것이 아닙니다. **257칸이 추측으로 채워져 있습니다.** 행 수는 같아도 데이터의 무게가 다릅니다.

**키워드** `numeric_only=True` 숫자 컬럼만 골라 계산

**정의** `median()`이나 `mean()`에 넘기면 **글자 컬럼을 빼고 숫자 컬럼만** 계산합니다.

**형태** `df.median(numeric_only=True)`

```
# 없으면 글자 컬럼에서 오류가 나거나 경고가 뜬다
print(base_8.median(numeric_only=True).head(4))
```

---

```
▶ 불량여부    0.00
▶ 사이클시간  16.70
▶ 성형사이클   16.72
▶ 배럴온도1    210.00
▶ dtype: float64
```

**왜 쓰나** 이 데이터에는 측정시각과 사출기라는 **글자 컬럼**이 있습니다. 글자의 중앙값이라는 것은 없으니, 그대로 두면 판다스가 곤란해집니다. `numeric_only=True`가 그것을 알아서 걸러 줍니다.

**응용** `fillna()`에 **Series**를 넘기면 **컬럼 이름끼리 짝을 맞춰** 각 컬럼을 자기 중앙값으로 채웁니다. 값 하나를 넘기는 것과 완전히 다른 동작입니다.

```
# 값 하나를 넘기면 모든 컬럼이 그 값으로 채워진다
a = base_8.fillna(0)

# Series를 넘기면 컬럼마다 다른 값으로 채워진다
b = base_8.fillna(base_8.median(numeric_only=True))

print("0으로 :", a["전환압력"].min())
print("중앙값으로:", round(b["전환압력"].min(), 2))
```

---

```
▶ 0으로 : 0.0          ← 1026 단위 컬럼에 0이 68개 섞임
▶ 중앙값으로: 1026.21
```

## 자주 하는 실수

`fillna(0)`은 **거의 언제나 잘못된 선택**입니다. 1026 근처의 압력 컬럼에 0을 68개 넣으면 평균이 무너집니다. 어제 배운 **위장 결측**을 스스로 만들어 넣는 셈입니다. 0은 그 값이 진짜로 "없음"을 뜻할 때만 씁니다.

## 4-2. 실습 9 — 파이프라인으로 묶고 저장

마지막 실습은 실습 3(컬럼 제거)과 실습 5(중앙값 대체)를 하나의 흐름으로 잇는 것입니다. 그리고 결과를 파일로 남깁니다.



Practice/15\_02\_example.py — 실습 9

```
df_9 = pd.read_csv("Data/15_02_사출성형_공정.csv", encoding="utf-8")

# ② 결측 비율 40% 초과 컬럼 제거
na_rate_9 = df_9.isna().sum() / len(df_9)
drop_cols_9 = na_rate_9[na_rate_9 > 0.4].index.tolist()
clean_9 = df_9.drop(columns=drop_cols_9)

# ③ 남은 결측을 중앙값으로 채우기
clean_9 = clean_9.fillna(clean_9.median(numeric_only=True))
print("최종 shape:", clean_9.shape, "| 남은 결측:", clean_9.isna().sum().sum())

# ④ 새 이름으로 저장 — 원본은 보존
out_path_9 = os.path.join("Data", "15_02_사출성형_공정_clean.csv")
clean_9.to_csv(out_path_9, index=False, encoding="utf-8")

# 저장한 파일을 다시 읽어 검증
check_9 = pd.read_csv(out_path_9, encoding="utf-8")
print("다시 읽은 shape:", check_9.shape, "| 남은 결측:", check_9.isna().sum().sum())
```

▶ 최종 shape: (250, 20) | 남은 결측: 0  
▶ 다시 읽은 shape: (250, 20) | 남은 결측: 0

### 개념

마지막 두 줄이 이 실습의 진짜 핵심입니다. **저장한 파일을 다시 읽어서 확인하는 것.** 저장이 제대로 됐는지, 인코딩이 깨지지 않았는지, 결측이 정말 0인지를 확인합니다. 어제 배운 **"확인으로 시작해 확인으로 끝난다"**가 여기서 완성됩니다. 제출 코드가 이 검증 단계를 스스로 넣은 것은 좋은 습관입니다.

**메서드** `.to_csv()` 데이터프레임을 CSV 파일로 저장

**정의** 처리 결과를 파일로 내보냅니다. 원본을 덮어쓰지 말고 새 이름으로 저장하는 것이 원칙입니다.

**형태** `df.to_csv("경로.csv", index=False, encoding="utf-8")`

```
clean_9.to_csv("Data/15_02_사출성형_공정_clean.csv", index=False, encoding="utf-8")
```

**왜 쓰나** 전처리는 시간이 오래 걸리는 작업입니다. 매번 처음부터 다시 돌릴 수 없으니 결과를 파일로 남깁니다. 다음 단계는 이 깨끗한 파일에서 시작합니다.

**응용** `index=False`가 중요합니다. 빼먹으면 판다스가 만든 행 번호가 새 컬럼으로 저장되어, 다시 읽을 때 쓸데없는 `Unnamed: 0` 컬럼이 생깁니다.

```
# index=False 없이 저장하면
clean_9.to_csv("/tmp/bad.csv", encoding="utf-8")
print(pd.read_csv("/tmp/bad.csv").columns[:3].tolist())

# index=False 를 붙이면
clean_9.to_csv("/tmp/good.csv", index=False, encoding="utf-8")
print(pd.read_csv("/tmp/good.csv").columns[:3].tolist())
```

---

- ▶ ['Unnamed: 0', '측정시각', '불량여부'] ← 쓸데없는 컬럼
- ▶ ['측정시각', '불량여부', '사이클시간']

#### 주의

**원본 파일 이름을 그대로 쓰면 안 됩니다.** 처리가 잘못됐을 때 되돌릴 방법이 사라집니다. `_clean` 같은 꼬리를 붙여 새 파일로 만드십시오. 실무에서는 **원본 보관 + 처리 기록 남기기**가 기본입니다.

## 치트시트

오늘 나온 모든 도구를 한 표에

## A-1. 제거 — dropna 계열

| 도구                     | 문법                                   | 하는 일           | 이 데이터 결과   |
|------------------------|--------------------------------------|----------------|------------|
| <code>.dropna()</code> | <code>df.dropna()</code>             | 결측 있는 행 삭제     | 250 → 76행  |
| <code>axis=1</code>    | <code>df.dropna(axis=1)</code>       | 결측 있는 열 삭제     | 22 → 10열   |
| <code>how="all"</code> | <code>df.dropna(how="all")</code>    | 모든 값이 빈 행만 삭제  | 250 → 250행 |
| <code>thresh=</code>   | <code>df.dropna(thresh=20)</code>    | 값이 20개 이상인 행만  | 250 → 162행 |
| <code>subset=</code>   | <code>df.dropna(subset=["A"])</code> | 지정 컬럼이 빈 행만 삭제 | 250 → 250행 |
| <code>.drop()</code>   | <code>df.drop(columns=[...])</code>  | 컬럼을 이름으로 삭제    | 22 → 20열   |

## A-2. 대체 — fillna 계열

| 방법     | 문법                                                      | 언제 쓰나                |
|--------|---------------------------------------------------------|----------------------|
| 상수     | <code>s.fillna(0)</code>                                | 그 값이 진짜 없음을 뜻할 때만    |
| 평균     | <code>s.fillna(s.mean())</code>                         | 숫자 · 고른 분포 · 이상치 없음  |
| 중앙값    | <code>s.fillna(s.median())</code>                       | 숫자 · 치우친 분포 · 이상치 있음 |
| 최빈값    | <code>s.fillna(s.mode()[0])</code>                      | 글자 · 분류 데이터          |
| 앞뒤 값   | <code>s.ffill().bfill()</code>                          | 시계열 — 정렬이 먼저         |
| 그룹별    | <code>gb.transform(lambda s: s.fillna(s.mean()))</code> | 그룹마다 평균이 다를 때        |
| 컬럼별 일괄 | <code>df.fillna(df.median(numeric_only=True))</code>    | 숫자 컬럼 전체를 한 번에       |

### A-3. 복사해 쓰는 결측 처리 골격

데이터를 받으면 이 순서로

```
import pandas as pd, os

# 1) 위장 결측을 아는 만큼 처음부터 NaN으로
df = pd.read_csv("파일.csv", encoding="utf-8", na_values=[ -999, 999])

# 2) 확인 — 개수와 비율을 한 표로
report = pd.DataFrame({
    "결측수": df.isna().sum(),
    "결측률(%)": (df.isna().mean() * 100).round(1),
})
print(report[report["결측수"] > 0].sort_values("결측률(%)", ascending=False))

# 3) 판단 — 40% 넘는 컬럼은 제거
na_rate = df.isna().sum() / len(df)
drop_cols = na_rate[na_rate > 0.4].index.tolist()
print("제거 대상:", drop_cols)
df = df.drop(columns=drop_cols)

# 4) 처리 — 남은 결측은 중앙값으로
df = df.fillna(df.median(numeric_only=True))

# 5) 검증 후 새 이름으로 저장
print("최종:", df.shape, "· 남은 결측:", df.isna().sum().sum())
df.to_csv("파일_clean.csv", index=False, encoding="utf-8")
```

#### 주의

이 골격의 4번 단계는 **컬럼 특성을 따지지 않고 한꺼번에 중앙값으로 채웁니다**. 편하지만 정석은 아닙니다. 실전에서는 **컬럼마다 종류와 분포를 확인하고 알맞은 방법을 골라야** 합니다. 글자 컬럼은 최빈값, 시계열은 앞뒤 값, 그룹 특성이 뚜렷하면 그룹별 평균입니다. 이 골격은 **첫 초안을 빠르게 만드는 용도**로만 쓰십시오.

### A-4. 오늘의 판단 기준

| 질문        | 기준               | 다음 행동     |
|-----------|------------------|-----------|
| 이 컬럼을 살릴까 | 결측률 <b>5% 미만</b> | 대체로 살리기   |
|           | 결측률 5 ~ 30%      | 중요도 보고 결정 |

| 질문         | 기준                         | 다음 행동                       |
|------------|----------------------------|-----------------------------|
|            | 결측률 <b>40% 이상</b>          | 제거 고민                       |
| 제거할까 대체할까  | 제거 시 남는 행이 <b>절반 아래</b> 인가 | 절반 아래면 대체로                  |
| 평균일까 중앙값일까 | 두 값의 <b>차이가</b> 큰가         | 크면 중앙값 · 이상치 있으면 무조건<br>중앙값 |
| 그룹별로 채울까   | 그룹별 평균이 <b>실제로</b> 다른가     | 거의 같으면 전체 평균으로 충분           |

## 개념

마지막 줄이 오늘 실습 7에서 배운 것입니다. **기법을 안다고 무조건 쓰는 것이 아닙니다.** 그룹별 대체는 좋은 기법이지만, 이 데이터의 사출기별 감압시간 평균은  $0.322179 \cdot 0.322368 \cdot 0.322400$ 으로 **소수 넷째 자리까지** 같습니다. 쓰나 마나입니다. **쓰기 전에 그룹별 평균을 먼저 찍어 보는 것이** 판단의 시작입니다.

## 오류·함정 사전

자주 걸리는 함정 · 교안 대조

## B-1. 오늘 걸리기 쉬운 함정 여섯 가지

| X 이렇게 하면                            | 무슨 일이                       | ✓ 바로잡기                                |
|-------------------------------------|-----------------------------|---------------------------------------|
| <code>df.dropna()</code>            | 저장 안 하면 원본은 그대로             | <code>clean = df.dropna()</code>      |
| <code>thresh=0.9</code>             | 비율이 아니라 개수라 의도와<br>다름       | <code>thresh=int(n_cols*0.9)</code>   |
| <code>fillna(0)</code>              | 1026 단위 컬럼에 0이 들어가<br>평균 붕괴 | <code>fillna(median)</code>           |
| <code>s.mode()</code>               | Series라 그대로 못 씀             | <code>s.mode()[0]</code>              |
| <code>ffill()</code> — 정렬 없이        | 아무 관계 없는 행의 값을 끌어옴          | <code>sort_values(시간)</code> 먼저       |
| <code>to_csv()</code> — index 옵션 없이 | Unnamed: 0 컬럼이 생김           | <code>to_csv(..., index=False)</code> |

## 자주 하는 실수

첫 줄이 결측 처리에서 압도적으로 흔한 실수입니다. `dropna`도 `fillna`도 새 데이터프레임을 만들어 돌려줄 뿐 원본을 바꾸지 않습니다. 결과를 변수에 담지 않으면 화면에는 처리된 듯 보이지만 원본은 그대로입니다. 교안의 표현대로 "처리했는데 왜 안 바뀌지"의 심중팔구가 이것입니다.

## B-2. inplace를 쓰지 않는 이유

예전 코드에는 `df.dropna(inplace=True)`처럼 원본을 직접 바꾸는 옵션이 자주 보입니다. 교안은 이것을 쓰지 말라고 명확히 적었습니다.

| 방식                          | 동작                                   | 문제                                             |
|-----------------------------|--------------------------------------|------------------------------------------------|
| X 결과 미저장                    | <code>df.dropna()</code>             | 화면엔 처리된 듯 보이지만 원본 그대로                          |
| X <code>inplace=True</code> | <code>df.dropna(inplace=True)</code> | 결과를 또 변수에 담으면 <b>None</b> 이 됨 · 최신 판다스가 권장 안 함 |
| ✓ 변수에 담기                    | <code>clean = df.dropna()</code>     | 원본 보존 + 결과 명확                                  |

### 주의

`inplace=True`의 진짜 문제는 **원본을 되돌릴 수 없다**는 점입니다. 처리가 잘못됐을 때 파일을 다시 읽어야 합니다. 그리고 `clean = df.dropna(inplace=True)`처럼 쓰면 `clean`에 **None**이 담깁니다. 원본을 바꾸는 함수는 아무것도 돌려주지 않기 때문입니다. **새 변수에 담은 습관 하나로 두 문제가 다 해결됩니다.**

## B-3. 교안 예상 결과 대조

| 교안        | 교안이 말한 것                                  | 실제 실행 결과                  |
|-----------|-------------------------------------------|---------------------------|
| 15_02 p4  | dropna 후 <b>250행 → 76행</b>                | <b>일치</b>                 |
| 15_02 p5  | dropna(axis=1) 후 <b>22열 → 10열</b>         | <b>일치</b>                 |
| 15_02 p7  | thresh=20 → <b>162행 유지</b>                | <b>일치</b>                 |
| 15_02 p14 | 실습 2 — "완전 결측 행만 삭제는 거의 유지, 임계값 20은 162행" | <b>일치</b>                 |
| 15_02 p15 | 실습 3 — "40% 초과 두 컬럼 제거 → 250×20"          | <b>일치</b>                 |
| 15_02 p16 | 실습 4 — "행삭제 손실 약 70%, thresh 손실 약 35%"    | 69.6% · 35.2% — <b>일치</b> |
| 15_02 p39 | 실습 8 — "제거 버전 110행, 대체 버전 250행"           | <b>일치</b>                 |

### 개념

**오늘은 교안 숫자가 전부 맞습니다.** 지난 회차들에서 라벨이 어긋나거나 다른 데이터 기준으로 적힌 경우가 잦았던 것과 대조적입니다. 그렇다고 확인을 생략하면 안 됩니다. **맞는 것을 확인하는 데도 같은 시간이 듭니다.** 그리고 틀린 곳이 한 군데라도 있으면 전부 다시 봐야 하니, 애초에 다 확인해 두는 편이 빠릅니다.

## 실습 하이라이트와 다음 시간

핵심 결론 정리 · 이상치 단위 예고

## C-1. 오늘 얻은 여섯 가지 결론

| # | 결론                      | 근거                                  |
|---|-------------------------|-------------------------------------|
| 1 | 삭제만으로는 데이터를 지킬 수 없습니다.  | 250행 → 76행, 손실 69.6%                |
| 2 | thresh가 손실을 절반으로 줄입니다.  | 69.6% → 35.2% (162행 보존)             |
| 3 | 채우면 분포가 뽀족해집니다.         | 최대사출압 std 8.51 → 7.42, 75% 분위수도 내려감 |
| 4 | 평균으로 채우면 평균은 그대로입니다.    | 원본 1241.672 = 평균대체 후 1241.672       |
| 5 | 그룹별 대체가 늘 의미 있는 건 아닙니다. | 사출기별 감압시간 평균이 소수 넷째 자리까지 동일         |
| 6 | 대체판 250행 중 434칸이 추측입니다. | 행 수는 같아도 데이터의 무게가 다름                |

## 설비 적용

6번이 오늘 가장 중요한 태도입니다. 제거판 110행과 대체판 250행 중 무엇이 나은지는 데이터만 봐서는 정할 수 없습니다. 뒤에 어떤 분석을 할지에 달렸습니다. 통계 요약이 목적이면 대체판이 낫고, 개별 사이클을 하나하나 추적하는 분석이라면 추측으로 채운 행이 오히려 방해가 됩니다. 그래서 실무에서는 원본 보관 + 처리 기록이 기본입니다.

## C-2. 오늘 푼 실습 정리

| 실습  | 내용           | 핵심 결과                                              |
|-----|--------------|----------------------------------------------------|
| 1·2 | dropna 옵션 조절 | 250→76 · how=all 그대로 · thresh20 162행 · subset 250행 |
| 3   | 비율 기준 컬럼 제거  | 40% 초과 2개(최대사출속도 · 감압시간) → 250 × 20                |
| 4   | 삭제 손실 비교     | 행삭제 69.6% · thresh 35.2% — 표로 정리                   |
| 5   | 평균 · 중앙값 대체  | 차이 3.267 — 거의 같아 어느 쪽도 무방                          |
| 6   | 최빈값 · 앞뒤 값   | 사출기 결측 0건 · 전환압력 68건을 ffill로 해결                    |
| 7   | 그룹별 대체       | 그룹 평균이 거의 같아 효과 없음 — transform 문법 학습               |

| 실습 | 내용          | 핵심 결과                                                 |
|----|-------------|-------------------------------------------------------|
| 8  | 제거 vs 대체 비교 | 제거판 110행(44%) · 대체판 250행(100%)                        |
| 9  | 종합 처리와 저장   | 40% 초과 제거 → 중앙값 채움 → <code>_clean.csv</code> 저장 → 재검증 |

### C-3. 다음 시간 예고 — 16\_01 이상치 개념과 사분위수

결측치 단원이 끝나고 **이상치 단원**이 시작됩니다. 결측치 **값이 비어 있는** 문제였다면, 이상치는 **값은 있는데 동떨어진** 문제입니다.

| 구분   |                                           |                                                 |
|------|-------------------------------------------|-------------------------------------------------|
| 상태   | 값이 비어 있음                                  | 값은 있으나 동떨어짐                                     |
| 표시   | NaN                                       | 채워진 숫자 (예: 250도)                                |
| 찾는 법 | <code>isna()</code>                       | <b><code>isna()</code>로 못 찾을 — 분포 계산 필요</b>     |
| 도구   | <code>dropna</code> · <code>fillna</code> | <code>quantile</code> → IQR → <code>clip</code> |

#### 연결

그런데 **처리 단계에서는 둘이 만납니다**. 이상치 처리의 한 방법이 **그 값을 일단 NaN으로 바꾼 뒤 채우는 것**이기 때문입니다. 오늘 배운 `fillna()`가 다음 시간에 이상치 처리 도구로 그대로 쓰입니다. 찾는 법은 달라도 고치는 법은 같습니다.

그리고 다음 시간에 배울 **사분위수와 IQR**은 20일차·21일차에 배운 중앙값의 연장선입니다. 중앙값이 데이터를 절반으로 나누는 선이라면, 사분위수는 **네 등분하는 세 개의 선**입니다. 이미 아는 개념이 확장되는 것이라 새로 외울 것이 많지 않습니다.

### C-4. 보너스 — 판다스 말고 다른 도구들

수업 끝 무렵 강사님이 **판다스 외의 데이터프레임 도구 두 가지**를 소개하셨습니다. 교안 차시 내용은 아니지만 알아 둘 만합니다.

| 도구            | 성격                | 장점                   | 단점                  |
|---------------|-------------------|----------------------|---------------------|
| <b>Pandas</b> | 파이썬 데이터 분석의 표준    | 가장 넓은 생태계 · 방대한 레퍼런스 | 싱글 코어라 대용량에서 느림     |
| <b>Polars</b> | Rust 기반 고속 데이터프레임 | 멀티스레딩 · 메모리 효율       | 문법이 달라 러닝 커브        |
| <b>DuckDB</b> | 분석용 임베디드 SQL 엔진   | 서버 없이 파일을 SQL로 바로 조회 | 메서드 체이닝이 아닌 SQL 문자열 |

Python/31\_dataframe\_tools.py

# 같은 작업을 세 도구로 — Electronics 필터 → 매출 계산 → 카테고리별 합계

```
# [Pandas]
f = df_pd[df_pd["category"] == "Electronics"].copy()
f["total_sales"] = f["quantity"] * f["unit_price"]
res = f.groupby("category").agg(total_sales=("total_sales", "sum")).reset_index()
```

# [Polars] — 표현식 기반 파이프라인

```
res = (pl.read_csv(CSV)
      .filter(pl.col("category") == "Electronics")
      .with_columns((pl.col("quantity") *
pl.col("unit_price")).alias("total_sales")))
      .group_by("category").agg(pl.col("total_sales").sum()))
```

# [DuckDB] — 파일 경로를 테이블처럼 직접 쿼리

```
res = duckdb.sql(f"SELECT category, SUM(quantity * unit_price) AS total_sales "
f"FROM '{CSV}' WHERE category = 'Electronics' GROUP BY "
category").df()
```

▶ 세 도구 모두 동일한 결과: total\_sales 1926.0 / total\_qty 6

### 주의

이 코드를 실행하려면 **설치가 필요합니다**. 강사님도 수업 중에 `pip install duckdb`를 직접 돌려 보이셨습니다. `pip install polars duckdb`로 두 개를 함께 설치하면 됩니다. 설치 없이 실행하면 `ModuleNotFoundError`가 납니다.

지금 단계에서 갈아탈 필요는 없습니다. 판다스가 표준이고 레퍼런스가 압도적으로 많습니다. 다만 이런 것들이 있다는 사실을 아는 것과 전혀 모르는 것은 다릅니다. 데이터가 수천만 행으로 커지면 판다스가 느려지고, 그때 선택지가 있다는 것을 알아야 찾아볼 수 있습니다.

## C-5. 스스로 확인하는 질문 여섯 개

| # | 질문                                        | 확인 지점                    |
|---|-------------------------------------------|--------------------------|
| 1 | <code>thresh=20</code> 은 무엇을 20개 이상 요구합니까 | 결측이 아니라 살아 있는 값 — 1-3    |
| 2 | <code>how="all"</code> 이 아무것도 안 지운 이유는    | 완전히 빈 행이 하나도 없어서 — 1-2   |
| 3 | <code>subset=</code> 은 언제 씁니까             | 정답 컬럼을 지킬 때 — 1-4        |
| 4 | 평균 대신 중앙값으로 채우는 이유는                       | 평균은 이상치에 오염됐을 수 있음 — 3-1 |

| # | 질문                                                    | 확인 지점             |
|---|-------------------------------------------------------|-------------------|
| 5 | <code>ffill()</code> 전에 반드시 할 일은                      | 시간 순으로 정렬 — 3-2   |
| 6 | <code>transform()</code> 이 <code>agg()</code> 와 다른 점은 | 원래 행 개수를 유지 — 3-3 |

## 오늘의 성취

버릴 것과 채울 것

## 연결

앞 표의 6번이 오늘 새로 배운 문법 중 가장 낯선 것입니다. `agg()`는 그룹당 한 줄로 줄이고, `transform()`은 **원래 행 개수를 그대로 유지**합니다. 결측을 채우려면 값이 **원래 자리에 놓여야** 하므로 `transform()`이 필요합니다. 이 차이 하나만 잡으면 그룹별 대체는 어렵지 않습니다.

오늘 배운 도구는 열 개 남짓입니다. `dropna`의 네 옵션, `fillna`의 여섯 방법, `transform`, `to_csv`. 그런데 도구 목록보다 중요한 것이 하나 있습니다.

## 개념

**모든 처리에는 대가가 있고, 그 대가를 숫자로 확인하는 것.**

행을 지우면 몇 %를 잃는지, 값을 채우면 분포가 얼마나 뒤틀려지는지. 오늘 실습이 계속 **손실률 · 보존율 · 표준편차 변화를 표로 만들**라고 요구한 이유가 이것입니다.

처리 방법을 고르는 것은 취향이 아닙니다. **69.6%와 35.2%를 나란히 놓고 보면 선택이 저절로 정해집니다.**

숫자를 안 만들면 감으로 고르게 됩니다.

한 걸음 더 나아가면, 오늘 배운 것은 **되돌릴 수 없는 결정을 다루는 법**입니다. 한 번 지운 데이터는 못 되돌리고, 한 번 채운 값은 진짜와 구분되지 않습니다. 그래서 **원본은 건드리지 않고 새 변수에 담고, 결과는 새 이름으로 저장하고, 무엇을 왜 했는지 기록**합니다. 이 세 가지 습관이 오늘 실습 전체에 깔려 있었습니다.

| 어제까지                                       | 오늘 더한 것                                                            | 다음에 만날 것                                                     |
|--------------------------------------------|--------------------------------------------------------------------|--------------------------------------------------------------|
| 빈 칸을 찾기                                    | 빈 칸을 지우거나 메우기                                                      | 튀는 값을 찾아 다루기                                                 |
| <code>isna</code> · <code>na_values</code> | <code>dropna</code> · <code>fillna</code> · <code>transform</code> | <code>quantile</code> · <code>IQR</code> · <code>clip</code> |
| 확인 단계                                      | 처리 단계                                                              | 새로운 문제 — 이상치                                                 |

마지막으로 오늘 확인한 숫자 하나만 다시 짚겠습니다. **250행 중 결측 없이 온전한 행은 76행, 30.4%뿐입니다.** 그리고 부실 컬럼 두 개를 먼저 빼도 110행밖에 안 남습니다. 이 데이터는 **삭제로는 답이 안 나오는 데이터**였고, 그 사실을 손실률 표 하나로 확인했습니다. **그 확인이 오늘 작업의 핵심**이었습니다.

— 23일차 정리 끝. 다음은 16\_01 이상치 개념과 사분위수입니다.